



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Cloud Computing: Análisis,
Diseño y Despliegue de un
Servicio Empresarial Seguro
en Microsoft Azure**



Autora: Karima Draflí Rico
en Universidad de Burgos — Junio 2026
Tutor: D. José Ignacio Santos Martín



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. José Ignacio Santos Martín, profesor del departamento de Ing. de Organización, área de OE.

Expone:

Que la alumna Dña. Karima Draffi Rico ha realizado el Trabajo final de Grado en Ingeniería Informática titulado “Cloud Computing: Análisis, Diseño y Despliegue de un Servicio Empresarial Seguro en Microsoft Azure”.

Y que dicho trabajo ha sido realizado por la alumna bajo la dirección de quien suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, junio de 2026

Vº. Bº. del Tutor:

D. José Ignacio Santos Martín

Resumen

Este Trabajo Fin de Grado aborda el problema de la gestión dispersa de solicitudes internas de tecnología. Cuando las peticiones se reciben por canales distintos resulta difícil aplicar criterios comunes, conocer su estado y mantener la trazabilidad de las decisiones.

Para estudiar este problema se ha desarrollado un prototipo funcional desplegado en Microsoft Azure. El sistema permite registrar solicitudes, relacionarlas con servicios y activos, clasificarlas mediante reglas, calcular su impacto y gestionar aprobaciones, responsables, escalados, plazos de atención, cierre y valoración.

La solución se ha implementado con una API Flask y un portal web alojados en Azure App Service, persistencia documental en Azure Blob Storage, secretos en Azure Key Vault, identidad administrada y una Logic App para recibir solicitudes desde sistemas externos. El desarrollo se organizó en cinco iteraciones y se validó mediante veintiséis pruebas automáticas, documentación OpenAPI, monitorización con Application Insights y análisis de código en SonarCloud.

El resultado no pretende sustituir a una plataforma ITSM comercial, sino demostrar de forma reproducible cómo integrar automatización, seguridad y operación cloud en un caso empresarial acotado. La memoria expone las decisiones tomadas, las alternativas consideradas y las limitaciones del prototipo, entre ellas el uso de un único blob JSON y la ausencia de control de concurrencia para una carga de producción.

Descriptores

Cloud computing, Microsoft Azure, Azure App Service, Azure Key Vault, Logic Apps, automatización de solicitudes, seguridad cloud.

Abstract

This Final Degree Project addresses the fragmented handling of internal IT requests. When requests arrive through different channels, applying common criteria, tracking their status, and preserving decision history become difficult.

To study this problem, a working prototype was developed and deployed on Microsoft Azure. The system registers requests, links them to services and assets, classifies them through deterministic rules, calculates business impact, and supports approval, assignment, escalation, SLA tracking, closure, and feedback.

The solution uses a Flask API and web portal hosted on Azure App Service, document persistence in Azure Blob Storage, secrets stored in Azure Key Vault, managed identity, and a Logic App for external systems. Development was organized into five iterations and validated through twenty-six automated tests, OpenAPI documentation, Application Insights monitoring, and SonarCloud analysis.

The result is not intended to replace a commercial ITSM platform. Its purpose is to demonstrate, in a reproducible and limited business case, how cloud automation, security, and operations can be integrated. The report discusses the decisions, alternatives, and limitations of the prototype, including its single JSON blob and the lack of production-grade concurrency control.

Keywords

Cloud computing, Microsoft Azure, Azure App Service, Azure Key Vault, Logic Apps, IT request automation, cloud security.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
2. Objetivos del proyecto	3
3. Conceptos teóricos	4
3.1. Cloud Computing	4
3.2. Microsoft Azure	6
3.3. Arquitectura basada en servicios	7
3.4. API REST	9
3.5. Seguridad y buenas prácticas cloud	10
3.6. DevOps y automatización	10
4. Técnicas y herramientas	12
4.1. Metodología de desarrollo	12
4.2. Lenguaje y framework backend	12
4.3. Control de versiones	13
4.4. Plataforma cloud	13
4.5. Automatización del despliegue	14
4.6. Calidad del código	15
4.7. Terraform	16
5. Aspectos relevantes del desarrollo del proyecto	17
5.1. Ciclo de vida del proyecto	17
5.2. Arquitectura del sistema	18
5.3. API REST implementada	20
5.4. Clasificador basado en reglas	23
5.5. Automatización del despliegue	26
5.6. Automatización empresarial con Logic App	26
5.7. Seguridad y gestión de secretos	26

5.8. Validación del prototipo	28
6. Trabajos relacionados	30
7. Conclusiones y Líneas de trabajo futuras	33
Bibliografía	35

Índice de figuras

3.1. Visión integral de la arquitectura cloud y de las evidencias del TFG. Fuente: elaboración propia.	8
5.1. Resumen final de sprints y puntos de historia en Zube. Fuente: elaboración propia a partir del tablero del proyecto.	18
5.2. Arquitectura final del prototipo desplegado en Microsoft Azure. Fuente: elaboración propia.	20
5.3. Centro operativo con SLA, carga, alertas y satisfacción tras la demostración. Fuente: captura propia.	24

Índice de tablas

3.1. Modelos principales de servicio cloud	5
3.2. Responsabilidad compartida en la solución cloud	5
3.3. Servicios Azure utilizados en la solución	7
4.1. Comparación de alternativas para aplicación, datos y servicios cloud	13
4.2. Comparación de alternativas para operación y seguimiento . . .	14
4.3. Herramientas utilizadas y justificación	15
5.1. Endpoints principales de la API	22
5.2. Reglas de negocio del clasificador automático	25
5.3. Escenarios de validación de las reglas de clasificación	25
5.4. Amenazas, controles aplicados y riesgos residuales	27
5.5. Niveles de validación y resultados del prototipo	29
6.1. Comparativa de trabajos y soluciones relacionadas	32

1. Introducción

Los equipos de tecnología reciben peticiones de acceso, cambios de configuración, altas de aplicaciones y avisos de servicio por canales muy distintos. Cuando esas peticiones terminan repartidas entre correos, hojas de cálculo y conversaciones, resulta difícil saber qué está pendiente, quién debe atenderlo o qué incidencias requieren prioridad.

El caso práctico de este TFG parte de ese problema. El objetivo no es reproducir todas las funciones de una plataforma comercial de gestión de servicios, sino construir un flujo acotado que pueda recorrerse de principio a fin: recibir una solicitud, validar sus datos, vincularla con un servicio y un activo, clasificarla, aprobarla cuando sea sensible, asignarla, controlar su SLA, cerrarla y recoger la valoración del solicitante.

La solución se ejecuta en Microsoft Azure. El portal y la API Flask comparten un App Service; las solicitudes se almacenan en Azure Cosmos DB como documentos JSON; Key Vault protege la clave de consulta; y la identidad administrada evita introducir credenciales de Azure en el código. Una Logic App añade un segundo canal de entrada para formularios o sistemas externos, mientras que Application Insights permite observar el servicio desplegado. El contrato OpenAPI permite revisar y ejecutar la interfaz REST desde el navegador.

La clasificación no depende de un servicio de aprendizaje automático. Se implementó como un conjunto de reglas deterministas sobre palabras clave, prioridades y tipos de solicitud. Esta elección permite revisar por qué se obtiene cada resultado y probar el comportamiento sin disponer de un histórico empresarial etiquetado.

El trabajo se dividió en seis sprints registrados en Zube. Las tarjetas, sus cambios de alcance y los commits de GitHub permiten reconstruir la evolución desde la preparación de Azure hasta la validación del despliegue y la mejora final de persistencia con Cosmos DB. El análisis de SonarCloud y veintisiete pruebas automáticas se utilizaron para corregir problemas concretos antes de generar los documentos finales.

Como materiales de evaluación se entregan el repositorio GitHub del proyecto, la memoria y anexos en PDF, el despliegue operativo en Azure

App Service, el tablero Zube utilizado para la planificación por sprints, el panel SonarCloud de calidad de código y las evidencias técnicas recogidas en la documentación del repositorio.

2. Objetivos del proyecto

El objetivo principal consiste en diseñar, desplegar y validar en Microsoft Azure un servicio que automatice el registro y la clasificación de solicitudes operativas de TI sin almacenar credenciales sensibles en el código.

Los objetivos específicos que concretan este propósito son los siguientes:

- Analizar las ventajas de las plataformas cloud en entornos empresariales.
- Diseñar una arquitectura segura basada en servicios cloud.
- Implementar una API REST utilizando Python y Flask.
- Desplegar la aplicación en Microsoft Azure.
- Gestionar secretos y configuraciones sensibles mediante Azure Key Vault.
- Automatizar el despliegue de la aplicación mediante scripts reproducibles con Azure CLI.
- Gestionar el proyecto utilizando metodologías ágiles y control de versiones.
- Implementar una clasificación automática y explicable mediante reglas de negocio.
- Analizar la calidad del código utilizando SonarQube/SonarCloud.
- Generar documentación técnica y funcional del sistema.

Estos objetivos conectan materias del grado que en el prototipo aparecen relacionadas: ingeniería del software para requisitos y pruebas, seguridad para secretos y permisos, sistemas distribuidos para la separación de servicios y gestión de proyectos para la planificación por iteraciones.

3. Conceptos teóricos

3.1. Cloud Computing

El cloud computing es un modelo tecnológico que permite acceder a recursos informáticos bajo demanda utilizando Internet. Este modelo reduce la necesidad de mantener infraestructuras físicas propias, permite escalar recursos dinámicamente y facilita que las organizaciones consuman capacidad de cómputo, almacenamiento, red y servicios gestionados según sus necesidades reales [4].

Entre las principales ventajas del cloud computing destacan:

- Escalabilidad.
- Alta disponibilidad.
- Pago por uso.
- Reducción de costes.
- Facilidad de mantenimiento.

Estas ventajas son especialmente relevantes en proyectos de alcance empresarial, ya que permiten pasar de una infraestructura rígida a un modelo más flexible, automatizable y orientado a servicios. En el caso de este TFG, el uso de la nube permite desplegar un prototipo accesible para el tribunal sin distribuir máquinas virtuales ni depender de una instalación local compleja.

Modelos de servicio

Los servicios cloud suelen clasificarse en distintos modelos según el nivel de responsabilidad que asume el proveedor y el nivel de control que conserva el usuario. La tabla 3.1 resume los modelos principales.

La solución desarrollada se apoya principalmente en un enfoque PaaS. Azure App Service permite ejecutar una API Flask y un portal web sin

Modelo	Responsabilidad principal del proveedor	Ejemplo en Azure
IaaS	Infraestructura básica: máquinas virtuales, red, discos y recursos de cómputo.	Azure Virtual Machines
PaaS	Plataforma gestionada para ejecutar aplicaciones sin administrar servidores.	Azure App Service
SaaS	Aplicación completa consumida como servicio por el usuario final.	Microsoft 365

Tabla 3.1: Modelos principales de servicio cloud

administrar directamente el sistema operativo, el balanceo interno ni la infraestructura física subyacente [9].

La tabla 3.2 resume cómo se reparte la responsabilidad entre el proveedor cloud y el equipo que desarrolla la aplicación. Esta distinción es importante porque el uso de servicios gestionados no elimina la responsabilidad de diseñar correctamente la aplicación, proteger los secretos y validar el despliegue.

Ámbito	Responsabilidad del proveedor cloud	Responsabilidad del proyecto
Infraestructura física	Centros de datos, hardware, red física y energía.	Selección de región y uso responsable de recursos.
Plataforma de ejecución	Mantenimiento del servicio gestionado App Service.	Configuración de runtime, variables de entorno y publicación de la aplicación.
Datos	Disponibilidad del servicio de almacenamiento.	Modelo de datos, privacidad, validación y persistencia correcta.
Identidad y secretos	Servicios como Key Vault y Managed Identity.	Definir permisos mínimos y evitar secretos en el repositorio.
Aplicación	Componentes gestionados de plataforma.	Código Flask, autenticación, pruebas y control de calidad.

Tabla 3.2: Responsabilidad compartida en la solución cloud

Modelos de despliegue

Además de los modelos de servicio, la nube puede organizarse mediante diferentes modelos de despliegue: nube pública, privada, híbrida o multicloud. Este proyecto utiliza nube pública, ya que los recursos se despliegan en Microsoft Azure y se consumen desde una suscripción Azure for Students. Para un prototipo académico, este enfoque permite validar una arquitectura real con costes controlados y acceso remoto.

3.2. Microsoft Azure

Microsoft Azure es la plataforma cloud de Microsoft orientada al desarrollo, despliegue y administración de aplicaciones empresariales [1]. Microsoft describe Azure como una plataforma para crear, desplegar y gestionar soluciones en la nube, con servicios de cómputo, datos, inteligencia artificial, seguridad y operaciones [5].

Los principales servicios utilizados en este proyecto son:

- Azure App Service.
- Azure Cosmos DB.
- Azure Blob Storage como persistencia inicial y origen de migración.
- Azure Key Vault.
- Managed Identity.
- Azure Monitor.

Servicios Azure utilizados

La tabla 3.3 relaciona los servicios Azure seleccionados con la responsabilidad concreta que cumplen dentro del prototipo.

La figura 3.1 resume la relación entre los servicios cloud seleccionados y las evidencias del proceso de desarrollo. La suscripción de Azure contiene el grupo de recursos del TFG, donde App Service ejecuta el portal y la API Flask, Key Vault protege el secreto utilizado como token Bearer, Cosmos DB conserva las solicitudes, Blob Storage queda como soporte de migración y Azure Monitor recoge telemetría. Fuera de Azure quedan las herramientas que apoyan el seguimiento y la calidad: GitHub, Zube y SonarCloud.

Servicio	Uso en el TFG	Justificación
App Service	Aloja la API Flask y el portal web.	Servicio PaaS adecuado para aplicaciones web y APIs REST.
Cosmos DB	Guarda cada solicitud como documento JSON.	Base de datos documental gestionada, adecuada para datos semiestructurados y consultas por solicitud.
Blob Storage	Conserva la persistencia inicial y sirve como origen de migración.	Almacenamiento sencillo y económico para objetos JSON.
Key Vault	Almacena el token de autenticación.	Evita credenciales embebidas en código fuente.
Managed Identity	Permite que App Service acceda a Key Vault.	Reduce la gestión manual de secretos y claves.
Azure Monitor	Apoya la revisión de estado y logs.	Facilita observabilidad básica del despliegue.

Tabla 3.3: Servicios Azure utilizados en la solución

Azure Cosmos DB es una base de datos NoSQL gestionada que permite trabajar con documentos JSON y contenedores particionados [10, 12]. Azure Blob Storage está orientado al almacenamiento de objetos y datos no estructurados, y se utilizó como persistencia inicial y origen de migración [19, 15]. Azure Key Vault centraliza secretos, claves y certificados, reduciendo la probabilidad de exponer información sensible en el código [8]. Las identidades administradas permiten que un recurso de Azure obtenga una identidad propia para acceder a otros servicios sin almacenar credenciales en la aplicación [18].

3.3. Arquitectura basada en servicios

Una arquitectura basada en servicios separa responsabilidades que evolucionan y se despliegan de forma distinta. En este proyecto la separación no pretende crear microservicios: busca que la interfaz, la lógica, la persistencia y la gestión de secretos tengan límites reconocibles.

Este enfoque facilita:

- Escalabilidad.
- Mantenimiento.

Arquitectura integral del TFG en Microsoft Azure

Portal, API, reglas de negocio, Cosmos DB, seguridad, automatización y evidencias de calidad

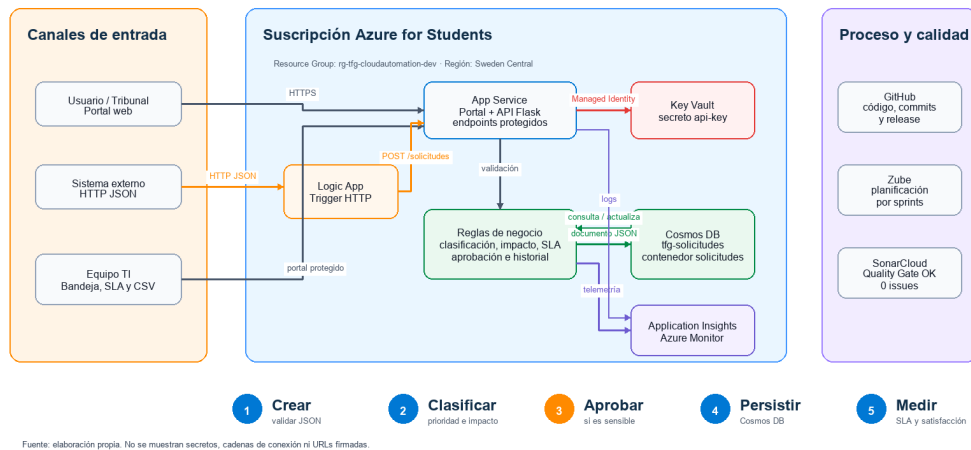


Figura 3.1: Visión integral de la arquitectura cloud y de las evidencias del TFG. Fuente: elaboración propia.

- Reutilización.
- Seguridad.

La arquitectura del proyecto sigue esta idea: el portal web se encarga de la interacción con el usuario, la API Flask concentra la lógica de negocio, el clasificador aplica reglas de análisis automático, Cosmos DB conserva las solicitudes y Key Vault protege el secreto usado por los endpoints protegidos. Esta separación facilita explicar el sistema, probarlo por partes y evolucionarlo en futuras versiones.

Plano de gestión y plano de datos

En una solución cloud conviene distinguir entre el plano de gestión y el plano de datos. El primero reúne las operaciones con las que se crean, configuran y supervisan recursos: alta del App Service, definición de variables de entorno, asignación de identidad o concesión de permisos. El segundo contiene las operaciones propias de la aplicación: registrar una solicitud, consultar su historial o guardar el documento en Cosmos DB. Mezclar ambos planos haría que una petición funcional necesitara permisos de administración sobre Azure, un acoplamiento innecesario y difícil de proteger.

En el prototipo, PowerShell y Azure CLI actúan sobre el plano de gestión durante el despliegue. La API, la Logic App y el portal operan en el plano de datos durante el uso normal. Managed Identity sirve de unión controlada entre ambos: la identidad se concede desde el plano de gestión, pero App Service la utiliza durante la ejecución para obtener el secreto autorizado. Esta separación permite aplicar privilegio mínimo y explica por qué el token de usuario no debe convertirse en una credencial de administración de la suscripción.

Persistencia documental y consistencia

La persistencia evolucionó durante el proyecto. La primera versión cloud utilizó Blob Storage con una colección JSON única; era sencilla y permitía inspeccionar el documento completo, pero obligaba a leer y escribir toda la colección en cada cambio. La versión actual utiliza Cosmos DB con API for NoSQL, manteniendo el formato JSON de las solicitudes y separando cada registro en un documento propio dentro del contenedor `solicitudes` [10, 11].

Esta evolución mejora la granularidad de los datos y responde a una limitación detectada en el diseño inicial: con un blob único, dos escrituras concurrentes podían partir de versiones distintas y sobrescribir cambios. Cosmos DB no convierte el prototipo en una plataforma ITSM completa, pero aporta una base documental más adecuada para consultas por solicitud, crecimiento incremental y particionado por atributos como el tipo de solicitud. La API sigue siendo la única responsable de validar, clasificar y modificar los datos; el portal y la Logic App nunca acceden directamente a la base de datos.

3.4. API REST

Una API REST permite la comunicación entre sistemas mediante peticiones HTTP.

En este proyecto se utilizan diferentes endpoints para:

- Crear solicitudes operativas.
- Consultar solicitudes e incidencias.
- Obtener métricas.

3.5. Seguridad y buenas prácticas cloud

La seguridad es un aspecto transversal en soluciones cloud. En este TFG se han aplicado tres decisiones principales: uso de HTTPS en el despliegue, separación de secretos mediante Key Vault y autenticación Bearer para las operaciones de consulta. Estas medidas no convierten el prototipo en una solución corporativa completa, pero sí demuestran buenas prácticas básicas de diseño seguro.

Además, el diseño se ha alineado con principios del Azure Well-Architected Framework, que propone pilares como seguridad, fiabilidad, eficiencia de rendimiento, optimización de costes y excelencia operativa [13]. En el contexto del proyecto, estos principios se reflejan en el uso de servicios gestionados, despliegue reproducible, recursos de bajo coste y documentación de evidencias.

Observabilidad frente a monitorización

La monitorización responde a preguntas conocidas, por ejemplo cuántas peticiones recibe el servicio o si se producen respuestas con error. La observabilidad busca además poder investigar comportamientos no previstos a partir de métricas, registros y trazas. Application Insights y Azure Monitor cubren esa necesidad básica en el proyecto: permiten comprobar actividad del App Service, localizar errores y relacionar el estado técnico con las pruebas funcionales [6].

El endpoint `/health` complementa estas herramientas, pero no las sustituye. Su respuesta confirma que la aplicación está disponible y muestra el modo de almacenamiento; no demuestra que todos los procesos empresariales funcionen. Por esa razón la validación combina tres niveles: salud técnica, pruebas automáticas del código y recorrido completo sobre el despliegue. Esta combinación evita considerar correcto un sistema únicamente porque su proceso web responde.

3.6. DevOps y automatización

DevOps es una metodología orientada a integrar desarrollo y operaciones con el objetivo de automatizar procesos y mejorar la calidad del software [2].

De las prácticas asociadas a DevOps, el proyecto utiliza las siguientes:

- Control de versiones y commits vinculados al seguimiento.

- Despliegue repetible mediante PowerShell y Azure CLI.
- Verificación automática de los endpoints publicados.
- Revisión externa de calidad con SonarCloud.

La versión final no mantiene un pipeline de integración continua. Los scripts de despliegue y verificación cubren la necesidad real del proyecto: repetir la publicación desde el equipo de desarrollo y dejar visibles las órdenes ejecutadas sobre Azure. Esta diferencia respecto al plan inicial se documenta en el seguimiento de sprints.

4. Técnicas y herramientas

4.1. Metodología de desarrollo

El trabajo se repartió en seis sprints. Cada iteración agrupó un resultado reconocible: preparación, API y seguridad inicial, despliegue, consolidación técnica, cierre documental y mejora final de persistencia con Cosmos DB.

Zube registró las historias, fechas y cambios de alcance. Su historial conserva también tres tarjetas retiradas del Sprint 3, una decisión que después se explicó como replanificación y no como trabajo oculto.

4.2. Lenguaje y framework backend

La aplicación backend se ha desarrollado utilizando Python junto con el framework Flask.

Flask permite construir APIs REST ligeras y fáciles de mantener. Su núcleo reducido deja explícitas las decisiones sobre rutas, validación, almacenamiento y autenticación, lo que resulta adecuado para un prototipo centrado en servicios cloud y no en las funciones de un framework web completo [20].

Justificación frente a otras alternativas

La selección tecnológica se realizó atendiendo al alcance, al tipo de carga y a la facilidad para demostrar cada objetivo del TFG. La tabla 4.1 recoge las principales alternativas consideradas; no establece que una herramienta sea universalmente superior, sino cuál encaja mejor con este prototipo.

La tabla 4.2 completa el análisis con las herramientas que sostienen el despliegue, la planificación y la calidad.

Necesidad	Decisión y justificación
API y portal web	Elección: Flask. Alternativa: Django. Django aporta ORM, administración y una estructura extensa, útiles en dominios relacionales complejos. Flask reduce componentes no utilizados y permite concentrar el prototipo en API, reglas de negocio e integración con Azure [3].
Persistencia cloud	Elección: Azure Cosmos DB con documentos JSON. Alternativas: Blob Storage, SQLite o Azure SQL. Los registros son semiestructurados y encajan con una base documental. Blob Storage se utilizó como primera persistencia cloud, pero Cosmos DB separa cada solicitud y resulta más adecuado para consultas y evolución del prototipo [10, 11].
Alojamiento	Elección: Azure App Service. Alternativas: máquina virtual o contenedor administrado manualmente. App Service abstrae sistema operativo, publicación y disponibilidad básica. Una máquina virtual añadiría mantenimiento que no forma parte del proceso estudiado.
Automatización	Elección: Azure Logic Apps. Alternativa: integración directa de cada sistema con la API. Logic Apps desacopla los sistemas externos y hace visible la orquestación. La API conserva validación, clasificación y persistencia para no duplicar reglas.
Secretos	Elección: Azure Key Vault con Managed Identity. Alternativa: variables o secretos gestionados manualmente. Key Vault centraliza el token y Managed Identity evita incorporar otra credencial de Azure al código.

Tabla 4.1: Comparación de alternativas para aplicación, datos y servicios cloud

4.3. Control de versiones

Git y GitHub conservaron el código y la documentación. Los commits separan, siempre que ha sido posible, cambios de aplicación, infraestructura y memoria para que resulte más sencillo relacionarlos con las tareas de Zube.

4.4. Plataforma cloud

La infraestructura cloud se basa en Microsoft Azure y en servicios gestionados documentados por Microsoft para aplicaciones web, bases de datos documentales, almacenamiento, monitorización y gestión de secretos [16, 11, 15, 17, 6].

Los servicios principales utilizados son:

Necesidad	Decisión y justificación
Despliegue	Elección: PowerShell y Azure CLI. Alternativa: pipeline CI/CD completo. El script es reproducible, auditable y suficiente para un único entorno académico. Un pipeline sería apropiado con varios equipos y promociones entre entornos.
Planificación	Elección: Zube. Alternativas: Azure Boards, Jira o GitHub Projects. Su integración con GitHub permitió conservar tablero, historias y seis sprints en un mismo historial.
Calidad	Elección: SonarCloud. Alternativa: SonarQube auto-hospedado. Proporciona métricas de seguridad, fiabilidad, mantenibilidad y duplicación sin administrar un servidor propio.

Tabla 4.2: Comparación de alternativas para operación y seguimiento

- Azure App Service.
- Azure Cosmos DB.
- Azure Storage como soporte de migración.
- Azure Key Vault.
- Azure Monitor.

La tabla 4.3 resume las herramientas empleadas y el motivo de su elección dentro del proyecto.

4.5. Automatización del despliegue

El despliegue se automatiza mediante el script `scripts/deploy-azure.ps1`, que utiliza Azure CLI para configurar App Service, Cosmos DB, Storage, Key Vault, Managed Identity y publicación ZIP de la aplicación. Esta decisión permite reproducir el despliegue desde un equipo local y documentar de forma explícita cada paso técnico aplicado sobre Azure.

El script automatiza:

- Comprobación de sesión en Azure.
- Configuración de secretos y variables de entorno.
- Asignación de identidad administrada.
- Publicación de la carpeta `src/` en Azure App Service.

Herramienta	Uso en el proyecto	Motivo de elección
Python y Flask	Implementación de la API REST y lógica de negocio.	Rapidez de desarrollo, claridad y ecosistema de bibliotecas.
Azure App Service	Despliegue del portal y la API.	Servicio PaaS adecuado para exponer aplicaciones web sin administrar servidores.
Azure Cosmos DB	Persistencia documental de solicitudes.	Mantiene el modelo JSON y evita concentrar todas las solicitudes en un único blob.
Azure Blob Storage	Persistencia inicial y origen de migración.	Almacenamiento sencillo para objetos JSON.
Azure Key Vault	Gestión del token de autenticación.	Separación de secretos respecto al código fuente.
Managed Identity	Acceso de App Service a Key Vault.	Evita credenciales manuales entre servicios Azure.
PowerShell y Azure CLI	Automatización del despliegue.	Reproducibilidad y trazabilidad de los pasos ejecutados.
Zube	Seguimiento de sprints y tablero Kanban.	Visibilidad del proceso y relación con GitHub.
SonarCloud	Análisis de calidad del código.	Métricas externas de mantenibilidad, fiabilidad, seguridad y duplicación.

Tabla 4.3: Herramientas utilizadas y justificación

4.6. Calidad del código

La calidad del código se analiza mediante SonarQube/SonarCloud [22].

En este proyecto se utiliza SonarCloud como servicio gestionado compatible con repositorios GitHub. Su uso permite obtener métricas objetivas de mantenibilidad, fiabilidad, seguridad y duplicidad sin desplegar un servidor SonarQube propio. El análisis se realiza desde el panel web de SonarCloud y complementa a las pruebas unitarias.

Esta herramienta permite detectar:

- Vulnerabilidades.
- Duplicidad.
- Complejidad excesiva.
- Posibles errores.
- Deuda técnica y problemas de mantenibilidad.

Los resultados del análisis se emplean como evidencia de calidad del código y como apoyo para priorizar mejoras antes de la entrega final.

4.7. Terraform

Terraform es una herramienta de infraestructura como código que describe recursos cloud de forma declarativa.

En este repositorio funciona como descripción versionada de la infraestructura Azure [7]. El despliegue que se ejecutó y validó se encuentra en el script PowerShell; por tanto, la memoria no atribuye a Terraform una automatización que no se utilizó en la publicación final.

5. Aspectos relevantes del desarrollo del proyecto

5.1. Ciclo de vida del proyecto

Los seis sprints muestran cómo cambió el producto desde la propuesta inicial hasta el despliegue validado y la persistencia final con Cosmos DB.

- Sprint 1, preparación y base: análisis del problema, estructura del repositorio, backlog y primeras decisiones de arquitectura.
- Sprint 2, implementación de módulos y dashboard: API y portal iniciales, persistencia, clasificación por reglas y primeras pruebas.
- Sprint 3, incidencias y despliegue: consolidación de solicitudes, validación, seguridad y despliegue funcional en Azure.
- Sprint 4, consolidación y documentación técnica: App Service, Storage, Key Vault, Managed Identity, infraestructura como código, evidencias y anexos.
- Sprint 5, finalización y defensa: ampliación empresarial, veintisiete pruebas, SonarCloud, OpenAPI, Cosmos DB, documentación y preparación de los entregables.

Esta relación utiliza los mismos nombres y alcance que la tabla A.1 del Anexo A. Las fechas previstas, los cierres reales y las replanificaciones se conservan allí y en Zube; este capítulo resume únicamente el incremento técnico producido en cada iteración. Los seis sprints quedaron cerrados el 30 de junio de 2026. El backlog suma 145 puntos estimados: 127 se cerraron dentro de las iteraciones y 18 se retiraron del Sprint 3 durante una replanificación documentada.

La figura 5.1 resume el cierre del proceso. Las barras muestran que no quedaron tarjetas ni puntos abiertos en ninguno de los seis sprints.

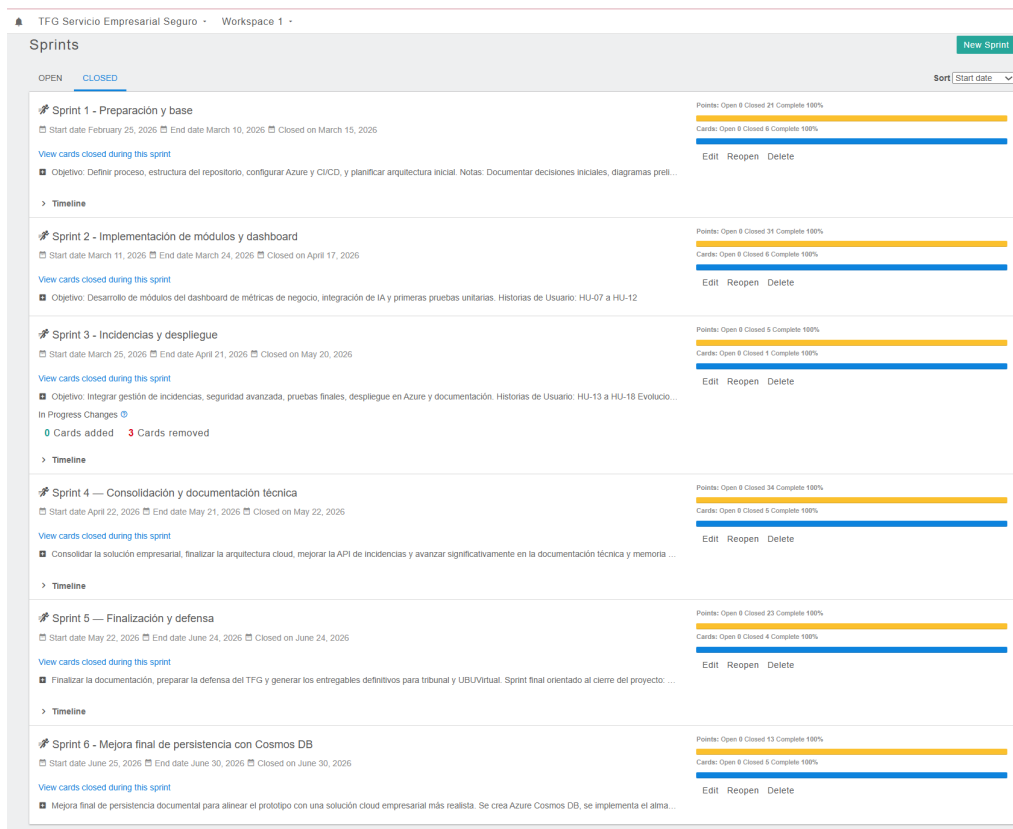


Figura 5.1: Resumen final de sprints y puntos de historia en Zube. Fuente: elaboración propia a partir del tablero del proyecto.

5.2. Arquitectura del sistema

La arquitectura implementada sigue un modelo de servicios gestionados en Microsoft Azure. Los componentes principales son:

- **Azure App Service:** hospeda la API Flask y el portal web empresarial en Python 3.11.
- **Azure Storage:** persiste las solicitudes operativas TI en un contenedor Blob privado.
- **Azure Key Vault:** almacena el token de autenticación de la API.
- **Azure Logic Apps:** automatiza la entrada de solicitudes externas mediante un trigger HTTP.

- **Application Insights y Azure Monitor:** permiten revisar telemetría, métricas y comportamiento operativo del servicio desplegado.
- **Scripts de despliegue:** automatizan la configuración y publicación en Azure mediante Azure CLI.

El flujo principal del sistema es el siguiente:

1. Un empleado envía una solicitud operativa TI desde el portal web, mediante la API REST o a través de la Logic App.
2. Azure App Service procesa la petición con Gunicorn.
3. La API valida los datos y clasifica automáticamente la solicitud.
4. El sistema genera una recomendación operativa para el equipo de TI.
5. Los datos se almacenan en Azure Cosmos DB como documentos JSON independientes.
6. Los secretos se obtienen desde Azure Key Vault mediante Managed Identity.
7. El endpoint de métricas y Application Insights permiten observar el estado agregado del sistema.

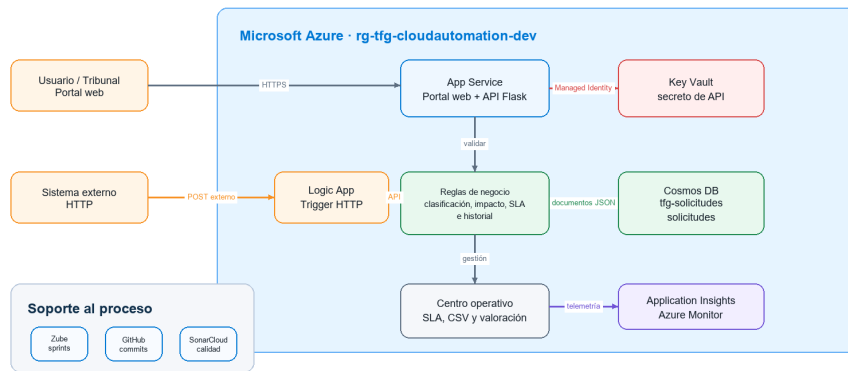
La figura 5.2 representa este flujo dentro de la arquitectura desplegada. Permite diferenciar los canales de entrada, la lógica ejecutada en App Service y los servicios gestionados responsables de secretos, persistencia y observabilidad.

Límites y dependencias entre componentes

La API es el punto de aplicación de las reglas del negocio. Tanto el portal como la Logic App llaman a la misma operación de creación; de este modo, añadir un canal de entrada no duplica validaciones ni clasificación. El portal se entrega como contenido estático desde el mismo App Service, una decisión que simplifica el prototipo y evita desplegar un segundo alojamiento. Esta unión sería revisable si el frontend necesitara un ciclo de publicación independiente, distribución mediante CDN o un equipo de desarrollo separado.

Arquitectura final de la solución en Microsoft Azure

Componentes reales del prototipo desplegado y relaciones principales



Fuente: elaboración propia. No se muestran secretos, cadenas de conexión ni URLs firmadas.

Figura 5.2: Arquitectura final del prototipo desplegado en Microsoft Azure. Fuente: elaboración propia.

Storage y Key Vault no son accesibles desde el navegador. La API actúa como frontera: valida el contenido antes de persistirlo y recupera el secreto mediante la identidad del App Service. La Logic App tampoco escribe directamente en el blob. Al conservar una única ruta de entrada se reduce el número de componentes capaces de alterar datos y se mantiene igual el resultado funcional con independencia del canal utilizado.

La dependencia más sensible es la disponibilidad del almacenamiento. Si Cosmos DB falla, la aplicación puede seguir respondiendo en el endpoint de salud, pero no debe confirmar una solicitud que no haya quedado persistida. La implementación propaga el error y evita devolver un identificador de éxito falso. En un entorno de producción se añadirían reintentos controlados, telemetría específica y una cola para absorber interrupciones temporales; en el prototipo se prioriza que el fallo sea visible y reproducible.

5.3. API REST implementada

La API desarrollada en `src/app.py` expone los siguientes endpoints:

- GET `/`: estado del servicio y versión.
- GET `/portal`, `/ayuda` y `/acerca`: portal, guía de uso e información verificable del proyecto.

- GET /docs y /openapi.json: documentación interactiva y contrato OpenAPI 3.0.
- GET /health: comprobación de salud y modo de almacenamiento.
- GET /catalogo: servicios, activos, entornos y responsables.
- POST /solicitudes: creación de solicitudes con clasificación automática.
- GET /solicitudes: listado con filtros por estado, prioridad y tipo.
- GET /solicitudes/<id>: consulta detallada de una solicitud.
- PATCH /solicitudes/<id>: actualización de estado, prioridad, responsable y notas.
- POST/solicitudes/<id>/aprobacion, /escalar y /valoracion: decisiones del flujo y feedback tras el cierre.
- GET /metricas, /operaciones y /informes/solicitudes.csv: indicadores, alertas e informe operativo.
- POST /demo/cargar: escenario idempotente con cinco casos de demostración.

La tabla 5.1 resume el contrato principal de la API, indicando el mecanismo de seguridad y la forma en que se ha validado cada operación.

Al crear una solicitud, el módulo `classifier.py` analiza el título y la descripción para asignar tipo de solicitud (*acceso, entorno, aplicación, configuración, incidencia*), prioridad (*baja, media, alta*), categoría (*seguridad, infraestructura, soporte, general*) y una recomendación operativa, devolviendo un valor de confianza asociado.

Después del registro, la bandeja del equipo TI permite gestionar el trabajo pendiente sin acceder directamente al almacenamiento. Cada actualización se incorpora al historial de la solicitud y las prioridades alta, media y baja reciben objetivos iniciales de cuatro, veinticuatro y setenta y dos horas, respectivamente. Estos datos alimentan el resumen de solicitudes en plazo, vencidas y cerradas.

Método	Ruta	Seguridad	Validación realizada
GET	/	Pública	Comprobación de estado general del servicio.
GET	/portal, /ayuda, /acerca	Pública	Recorrido del portal y materiales de demostración.
GET	/docs, /openapi.json	Pública	Revisión y ejecución del contrato REST.
GET	/health	Pública	Verificación de estado y modo de almacenamiento.
GET	/catalogo	Pública	Consulta de servicios, activos y entornos.
POST	/solicitudes	Pública con validación de entrada	Creación de solicitud y clasificación automática.
GET	/solicitudes	Token Bearer	Consulta protegida con filtros y paginación.
GET/PATCH	/solicitudes/<id>	Token Bearer	Detalle, historial y actualización controlada.
POST	/solicitudes/<id>	Token Bearer	Decisión y escalado con actor y motivo.
POST	/solicitudes/<id>/valoracion	Correo del solicitante	Feedback único sobre una solicitud cerrada.
GET	/metricas, /operaciones, CSV	Token Bearer	Métricas, alertas, SLA, satisfacción e informe.
POST	/demo/cargar	Token Bearer	Preparación idempotente del escenario de evaluación.

Tabla 5.1: Endpoints principales de la API

Ciclo operativo de una solicitud

El registro no se limita a guardar un formulario. Una solicitud nace en estado abierta, recibe un identificador estable y conserva la clasificación calculada. El equipo de TI puede asignar un responsable, ajustar la prioridad cuando dispone de más contexto y pasarla a *en proceso* o *cerrada*. Cada modificación incorpora actor, fecha y detalle al historial. La transición desde cerrada hacia abierta se rechaza para evitar reabrir trabajo sin una regla explícita; una futura versión podría incorporar una operación específica de reapertura con motivo obligatorio.

Los objetivos de cuatro, veinticuatro y setenta y dos horas no se presentan como un acuerdo de nivel de servicio contractual. Son umbrales operativos del prototipo que permiten demostrar cómo una prioridad afecta a la gestión

posterior. Las métricas comparan la fecha objetivo con el estado actual y distinguen trabajo en plazo, vencido y cerrado. El equipo puede así ordenar la bandeja por impacto y no únicamente por fecha de llegada.

Esta evolución aproxima el producto a una herramienta de trabajo diaria: el solicitante aporta el contexto inicial, el clasificador propone un tratamiento y el equipo conserva la decisión humana final. El sistema automático no cierra solicitudes ni asigna permisos; ayuda a ordenar información y deja trazabilidad sobre las acciones realizadas.

Demostración y feedback

Para que la evaluación no dependa de preparar manualmente datos antes de cada demostración se incorporó `POST /demo/cargar`. La operación requiere el token del equipo TI y crea cinco situaciones diferenciadas: acceso sensible pendiente de aprobación, incidencia crítica con SLA vencido, cambio en proceso, solicitud cerrada y caso escalado. Cada registro queda marcado como demostración y una segunda ejecución devuelve los existentes sin duplicarlos. Esta propiedad permite repetir la defensa conservando identificadores y métricas comprensibles.

El contrato se publica como OpenAPI 3.0 en `/openapi.json` y se presenta mediante Swagger UI en `/docs`. Desde esa vista se distinguen operaciones públicas y protegidas, se introduce temporalmente el Bearer token y se inspeccionan peticiones, respuestas y códigos HTTP. La documentación ejecutable reduce la distancia entre lo descrito y lo desplegado: los ejemplos llaman a la misma instancia de App Service que utiliza el portal.

El cierre incorpora un mecanismo de feedback deliberadamente limitado. Solo se admite una valoración de uno a cinco puntos, la solicitud debe estar cerrada y el correo debe coincidir con el remitente original. El resultado se añade al historial y el centro operativo calcula número de respuestas y promedio. No se presenta como estudio estadístico, pues el conjunto de demostración es pequeño; su función es cerrar el proceso y mostrar cómo una medida de calidad percibida puede integrarse con SLA, carga y alertas.

La figura 5.3 muestra el resultado conjunto después de recorrer el escenario de evaluación.

5.4. Clasificador basado en reglas

El clasificador de `src/classifier.py` utiliza palabras clave y reglas de negocio. Es un mecanismo determinista cercano a la inteligencia artificial

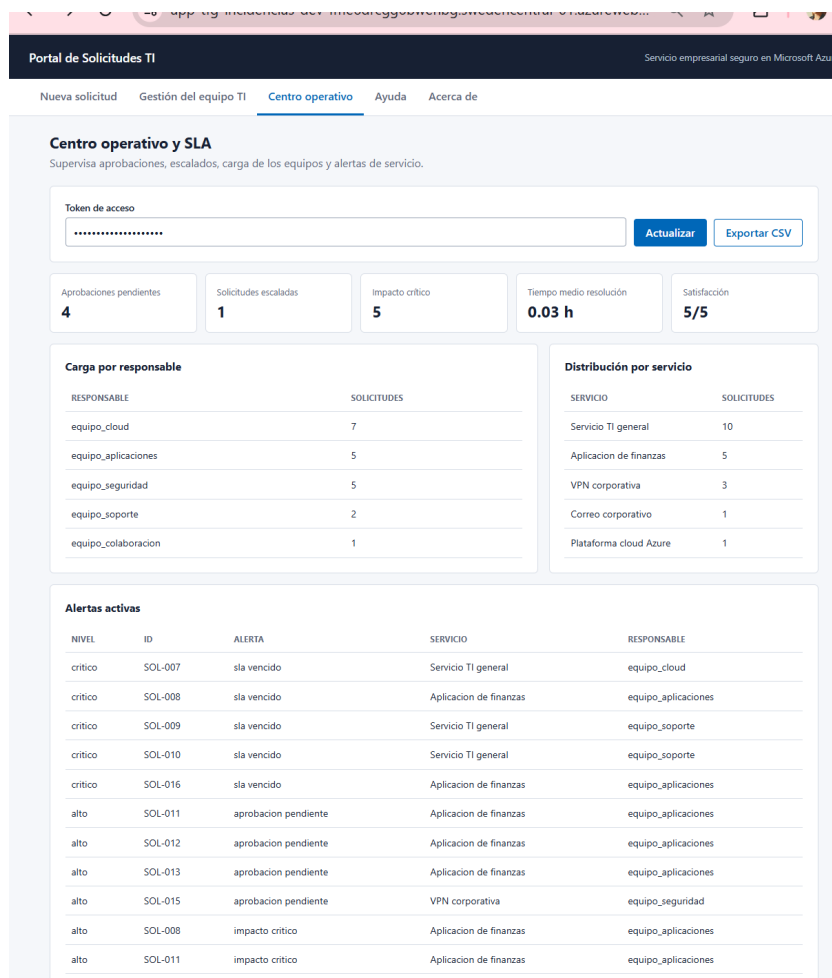


Figura 5.3: Centro operativo con SLA, carga, alertas y satisfacción tras la demostración. Fuente: captura propia.

simbólica, no un modelo entrenado. La distinción evita atribuirle capacidades que no tiene: el resultado puede explicarse siguiendo las puntuaciones, pero el sistema no aprende de nuevas solicitudes.

Las reglas se evalúan en un orden definido. Una prioridad o un tipo válidos indicados explícitamente por el usuario prevalecen sobre la inferencia. En ausencia de esos valores, se calculan puntuaciones por coincidencia de términos; la categoría con mayor puntuación se selecciona y, si no hay coincidencias, se utiliza la categoría general. Cuando dos categorías empatan, se aplica el orden estable seguridad, infraestructura y soporte. El sistema

produce una única clasificación y recomendación, por lo que no genera alertas paralelas ni ejecuta dos acciones incompatibles.

La tabla 5.2 resume las reglas implementadas en `src/classifier.py`.

Regla	Condición	Resultado
R-01	Prioridad manual incluida en baja, media o alta.	Se conserva la prioridad indicada con confianza máxima.
R-02	Dos o más términos críticos, o caída acompañada de un término crítico.	Prioridad alta y atención recomendada en menos de cuatro horas.
R-03	Un término crítico o dos términos de impacto medio.	Prioridad media y resolución en la siguiente ventana operativa.
R-04	No se cumple una regla de prioridad anterior.	Prioridad baja y planificación en el backlog.
R-05	Tipo manual válido o mayor puntuación de palabras del tipo.	Tipo de solicitud explícito o inferido; incidencia como valor de respaldo.
R-06	Mayor puntuación entre seguridad, infraestructura y soporte.	Categoría única; general si todas las puntuaciones son cero.
R-07	Combinación de tipo, categoría y prioridad.	Recomendación específica y plazo operativo asociado.

Tabla 5.2: Reglas de negocio del clasificador automático

Para validar el comportamiento con entradas distintas, se definieron los escenarios de la tabla 5.3. Estos casos se ejecutan automáticamente en la batería de pruebas.

Ejemplo de entrada	Tipo	Categoría	Prioridad
Intrusión y acceso no autorizado críticos	Incidencia	Seguridad	Alta
Error y fallo en servidor VPN	Incidencia	Infraestructura	Media
Ayuda para configuración manual de usuario	Configuración	Soporte	Baja
Consulta sobre calendario de vacaciones	Incidencia	General	Baja

Tabla 5.3: Escenarios de validación de las reglas de clasificación

5.5. Automatización del despliegue

El despliegue reproducible se concentra en `scripts/deploy-azure.ps1`. El script:

1. Comprueba la sesión de Azure CLI.
2. Obtiene la configuración de Azure Storage y Key Vault.
3. Habilita Managed Identity en Azure App Service.
4. Concede a la aplicación permisos sobre el secreto almacenado en Key Vault.
5. Configura variables de entorno y publica el código Python en Azure App Service.

5.6. Automatización empresarial con Logic App

La Logic App actúa como capa de orquestación para integrar la plataforma con otros sistemas de la empresa. En el prototipo, un sistema externo puede enviar una petición al trigger HTTP de la Logic App y esta invoca de forma controlada el endpoint `POST /solicitudes` de la API desplegada en App Service [14]. La API conserva la responsabilidad de validar datos, clasificar la solicitud, generar la recomendación y persistir el registro en Azure Cosmos DB.

Este flujo resulta útil para un escenario empresarial realista: un formulario corporativo, una herramienta de soporte, un buzón automatizado o un sistema de atención interna podrían crear solicitudes sin acceder manualmente al portal. La Logic App no sustituye al equipo de TI ni al backend, sino que automatiza la entrada de trabajo. Si la API determina que el servicio, el activo y el entorno requieren aprobación, el flujo registra la notificación al aprobador y la solicitud permanece bloqueada hasta recibir una decisión. La asignación, el escalado, el seguimiento y el cierre se realizan en la bandeja operativa del portal.

5.7. Seguridad y gestión de secretos

La autenticación de la API utiliza un token Bearer almacenado en Azure Key Vault. La App Service accede al secreto mediante Managed Identity,

evitando credenciales embebidas en el código fuente. El almacenamiento de solicitudes reside en un contenedor Blob con acceso privado.

Modelo de amenazas aplicado

El análisis de seguridad se acotó a las fronteras reales del prototipo. No se asumió que utilizar Azure eliminara riesgos de aplicación. Se revisaron entrada de datos, acceso a operaciones de gestión, exposición de secretos, almacenamiento y configuración de infraestructura. La tabla 5.4 resume los escenarios principales y el tratamiento adoptado.

Amenaza	Consecuencia	Control aplicado	Riesgo residual
Entrada malformada o excesiva	Datos incoherentes, errores o consumo innecesario.	Campos obligatorios, tipos permitidos y límites de longitud.	No se dispone de un firewall de aplicaciones ni limitación por usuario.
Consulta sin autorización	Exposición de solicitudes e información operativa.	Token Bearer en listado, detalle, actualización y métricas.	El token es compartido y no identifica individualmente al operador.
Secreto incluido en código	Filtración mediante Git o documentación.	Secreto en Key Vault, variables de entorno y exclusión de valores reales.	La rotación requiere una actuación administrativa.
Acceso directo al almacenamiento	Modificación o lectura fuera de las reglas de la API.	Contenedor privado y acceso desde el backend.	La cadena de conexión conserva privilegios amplios sobre la cuenta.
Tráfico sin cifrar	Interceptación de credenciales o datos.	HTTPS obligatorio en App Service y TLS en Storage.	No se han desplegado red privada ni puntos de conexión privados.
Cambio no trazable	Dificultad para atribuir una decisión operativa.	Historial con actor, fecha y comentario.	El actor declarado no se valida contra un directorio corporativo.

Tabla 5.4: Amenazas, controles aplicados y riesgos residuales

La principal limitación es la identidad. El Bearer token demuestra protección básica de endpoints y almacenamiento seguro de la credencial, pero no equivale a autenticación corporativa. Una evolución a Microsoft Entra ID permitiría identificar usuarios, asignar roles y revocar accesos individualmente. También sería necesario limitar el endpoint público de creación mediante identidad, cuota o protección frente a abuso antes de aceptar datos reales.

La información utilizada en las evidencias es ficticia. Aun así, una explotación empresarial tendría que definir política de conservación, derecho de acceso, minimización y borrado. La trazabilidad técnica no sustituye el análisis legal del tratamiento de datos personales.

5.8. Validación del prototipo

La validación se organizó en tres capas. La primera ejecuta veintisiete pruebas automáticas con almacenamiento temporal local y dobles de prueba para Azure; permite repetir casos sin modificar los datos desplegados. La segunda utiliza `verify-azure.ps1` y pruebas HTTP contra la URL pública para comprobar el recorrido real por App Service, Key Vault, Cosmos DB y Logic Apps. La tercera conserva capturas del portal, Azure Portal, Application Insights, Zube y SonarCloud para relacionar el resultado técnico con los recursos observados.

Las pruebas automáticas cubren creación, consulta, autenticación, validaciones de entrada, filtros, paginación, ciclo de vida, historial, persistencia, métricas y reglas de clasificación. La ampliación añade catálogo de servicios, cálculo de impacto, aprobación, escalado, centro operativo, exportación CSV, demostración idempotente, OpenAPI, valoración y consistencia entre procesos de App Service. Se incluyen caminos negativos: token incorrecto, campos vacíos, correo no válido, prioridad desconocida, paginación fuera de rango, transición no permitida, valoración repetida e identificador inexistente.

El resultado confirma el funcionamiento del caso práctico, pero no constituye una prueba de carga ni una certificación de seguridad. No se ha medido concurrencia elevada, recuperación regional, disponibilidad mensual o comportamiento con miles de solicitudes. Estas pruebas exigirían un entorno estable durante más tiempo, datos representativos y criterios de aceptación acordados con una organización usuaria.

La posibilidad de ejecutar el backend con fichero local, mantener compatibilidad con Blob Storage y desplegarlo con Cosmos DB también se utilizó como mecanismo de prueba. La lógica de la API permanece igual y la estrategia de persistencia se selecciona mediante configuración. Esta separación permite desarrollar sin consumir Azure y comprobar después la integración cloud sin mantener dos implementaciones funcionales diferentes.

Nivel	Comprobación	Resultado observado
Unidad e integración local	Veintisiete pruebas sobre API, almacenamiento, flujos y clasificador.	Todas finalizan correctamente con datos temporales.
Despliegue público	Página raíz, portal y <code>/health</code> .	Servicio accesible por HTTPS y almacenamiento en modo Azure.
Proceso empresarial	Catálogo, alta, impacto, aprobación, escalado, SLA, valoración e informe CSV.	Solicitud persistida y gestionada con trazabilidad completa.
Demostración y contrato	Escenario idempotente y operaciones descritas en OpenAPI.	Cinco casos reproducibles y documentación interactiva accesible.
Flujo externo	Trigger HTTP de Logic App, alta en API y notificación de aprobación.	Ejecución correcta y respuesta de solicitud creada.
Seguridad	Token Bearer, Key Vault, Managed Identity y blob privado.	Consultas protegidas y secreto fuera del repositorio.
Observabilidad	Application Insights, Azure Monitor y endpoint de salud.	Telemetría y métricas disponibles para inspección.
Calidad estática	Revisión del repositorio en SonarCloud.	Duplicación nula y análisis externo documentado.

Tabla 5.5: Niveles de validación y resultados del prototipo

6. Trabajos relacionados

Para situar el alcance del prototipo se tomaron como referencia dos grupos de soluciones. El primero incluye plataformas de gestión de servicios y soporte, como ServiceNow, Jira Service Management y Zendesk. El segundo reúne trabajos académicos centrados en arquitectura cloud y automatización del despliegue; entre ellos, Sanclemente Vilda estudia una arquitectura de referencia para microservicios mediante prácticas DevOps y GitOps [21].

Las plataformas comerciales parten de una necesidad parecida, pero ofrecen un alcance mucho mayor: catálogos de servicio, acuerdos de nivel de servicio, automatizaciones configurables, bases de conocimiento, inventario e integración con directorios corporativos. Su ventaja es la madurez funcional; su inconveniente para los objetivos del trabajo es que utilizar un producto terminado ocultaría buena parte del análisis de arquitectura, seguridad, persistencia y despliegue que se pretende demostrar.

El proyecto no intenta reconstruir una suite ITSM. Selecciona un recorrido pequeño pero completo: recepción de una solicitud, validación, clasificación explicable, persistencia, gestión por el equipo y observación del resultado. Esta reducción permite que las decisiones sean visibles en el código y en Azure. También deja fuera funciones que serían imprescindibles en producción, especialmente identidad individual, catálogo formal, notificaciones, búsqueda avanzada y gestión contractual de SLA.

Los trabajos académicos sobre cloud suelen concentrarse en comparar modelos de servicio, describir proveedores o desplegar una aplicación de ejemplo. La referencia de Sanclemente Vilda aporta una perspectiva más orientada a automatización e infraestructura, con tecnologías adecuadas para arquitecturas de microservicios. El presente TFG comparte la preocupación por reproducibilidad y operación, pero utiliza servicios PaaS más sencillos porque el dominio no necesita un clúster ni múltiples servicios desplegables. La diferencia no es solo tecnológica: aquí la infraestructura se evalúa como soporte de un proceso empresarial concreto.

La comparación no pretende evaluar experimentalmente los productos comerciales ni afirmar que el prototipo pueda sustituirlos. Sirve para identificar qué parte del problema se aborda en el TFG:

- recepción y consulta de solicitudes TI;
- clasificación explicable mediante reglas;
- despliegue real sobre servicios Azure;
- protección de secretos y acceso entre servicios;
- trazabilidad mediante pruebas, sprints y evidencias.

Se dejó fuera, de forma deliberada, la gestión completa de acuerdos de servicio, inventario, facturación, flujos de aprobación y directorio corporativo.

La aportación propia está en recorrer el ciclo completo con una solución acotada: requisitos, arquitectura, código, despliegue, seguridad, pruebas y evidencias. Esta perspectiva difiere tanto del uso de una herramienta comercial ya construida como de un trabajo centrado exclusivamente en infraestructura.

La comparación también ayuda a situar el clasificador. Las plataformas empresariales pueden incorporar modelos estadísticos o servicios de inteligencia artificial entrenados con grandes historiales. El proyecto utiliza reglas deterministas porque no dispone de un conjunto de solicitudes reales etiquetadas. Esta elección sacrifica aprendizaje automático, pero permite explicar cada resultado y comprobarlo con casos reproducibles. Incorporar un servicio de lenguaje solo sería justificable después de construir un conjunto de evaluación y comparar precisión, coste y tratamiento de datos.

Por tanto, el valor diferencial no reside en superar funcionalmente a las soluciones comerciales, sino en demostrar de forma auditable cómo se construye y despliega una base segura para ese tipo de proceso. El tribunal puede recorrer los requisitos, localizar su implementación, ejecutar pruebas, acceder al portal y revisar los recursos cloud que sostienen cada decisión.

Solución	Ámbito principal	Relación con el TFG	Diferencia de alcance
ServiceNow	Gestión de servicios empresariales	Comparte el tratamiento estructurado de solicitudes.	El TFG no cubre el catálogo ITSM completo.
Jira Service Management	Solicitudes y trabajo de equipos técnicos	Comparte el seguimiento de peticiones y estados.	El TFG se centra en construir y desplegar la arquitectura.
Zendesk	Atención y soporte	Comparte el canal de entrada y la organización de solicitudes.	El prototipo prioriza servicios Azure y seguridad entre componentes.
Trabajo de Sanclemente Vilda	Arquitectura DevOps/GitOps para micro-servicios	Aporta una referencia académica sobre automatización e infraestructura.	Su objeto son micro-servicios y GitOps, no la gestión de solicitudes.
Proyecto desarrollado	Gestión de solicitudes TI en Azure	Integra portal, API, reglas, seguridad, persistencia y observabilidad.	Es un prototipo académico con datos de prueba.

Tabla 6.1: Comparativa de trabajos y soluciones relacionadas

7. Conclusiones y Líneas de trabajo futuras

El resultado del TFG es un flujo funcional y desplegado, no solo un diseño de arquitectura. Una solicitud puede entrar desde el portal, la documentación OpenAPI o la Logic App, pasa por la misma validación y clasificación, se conserva en Cosmos DB y queda disponible para aprobación, asignación, escalado y seguimiento del SLA. Tras el cierre, el solicitante puede valorar la resolución y el centro operativo agrega la satisfacción junto con la carga y las alertas.

Las decisiones más útiles surgieron al reducir el alcance. Flask encajó mejor que un framework completo porque la API tiene pocas rutas y no necesita un modelo relacional. La persistencia evolucionó desde Blob Storage hacia Cosmos DB porque el modelo documental seguía siendo sencillo, pero convenía separar cada solicitud en un documento propio. El pipeline de GitHub Actions previsto al principio se descartó a favor de scripts PowerShell y Azure CLI que podían ejecutarse, revisar y demostrar desde el entorno disponible.

La parte de seguridad también dejó una conclusión concreta: trasladar una clave a Key Vault no basta si la aplicación necesita otra credencial para leerla. La identidad administrada resuelve esa relación entre App Service y Key Vault sin añadir secretos al repositorio. SonarCloud ayudó a revisar configuraciones de infraestructura y a distinguir entre vulnerabilidades reales, decisiones intencionadas y aspectos pendientes de una solución de producción.

El trabajo permitió practicar de forma conjunta los siguientes ámbitos:

- Seguridad cloud.
- Gestión de secretos.
- APIs REST.
- Automatización de procesos.
- Arquitecturas escalables.

- Clasificación automática de información operativa.
- Documentación y prueba interactiva de APIs mediante OpenAPI.
- Seguimiento de SLA y evaluación de satisfacción.

El prototipo cumple los objetivos planteados, aunque mantiene límites claros. La autenticación Bearer protege consultas técnicas, pero no sustituye la identidad individual de usuarios; Cosmos DB cubre mejor la persistencia documental del prototipo, aunque no sustituye un diseño completo de gobierno de datos, permisos por usuario y explotación analítica; y el clasificador es explicable y repetible, pero no aprende de casos anteriores. Estas limitaciones orientan las líneas de continuación.

Como líneas futuras se plantean:

- Evaluar un servicio de lenguaje solo cuando exista un conjunto de solicitudes etiquetadas con el que comparar su precisión frente a las reglas actuales.
- Incorporar autenticación de usuarios con Azure AD.
- Crear dashboards avanzados de monitorización y alertas operativas.
- Incorporar un área personal para que cada usuario consulte exclusivamente sus solicitudes.
- Incorporar notificaciones automáticas por correo o Teams cuando cambie el estado de una solicitud.
- Añadir políticas de retención, auditoría y archivado sobre los documentos almacenados en Cosmos DB.

Bibliografía

- [1] Azure Architecture Center, Microsoft Learn. Browse azure architectures, 2025. Consultado: marzo 2026.
- [2] Karthik Bojja. Scalable devops practices with azure: Automating secure kubernetes deployments. *IJIRSET*, 9(6), Junio 2020.
- [3] Django Software Foundation. Django at a glance, 2026. Documentación oficial, consultado junio 2026.
- [4] Microsoft. Microsoft azure: Cloud computing dictionary, 2025. Consultado: marzo 2026.
- [5] Microsoft Azure. What is azure?, 2026. Documentación oficial de Microsoft Azure, consultado junio 2026.
- [6] Microsoft Learn. Azure monitor, 2025.
- [7] Microsoft Learn. Terraform for azure, 2025.
- [8] Microsoft Learn. About azure key vault, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [9] Microsoft Learn. Azure app service overview, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [10] Microsoft Learn. Azure cosmos db documentation, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [11] Microsoft Learn. Azure cosmos db for nosql client library for python, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [12] Microsoft Learn. Azure cosmos db for nosql documentation, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [13] Microsoft Learn. Azure well-architected framework, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [14] Microsoft Learn. Call, trigger, or nest workflows with http endpoints in azure logic apps, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.

- [15] Microsoft Learn. Quickstart: Azure blob storage client library for python, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [16] Microsoft Learn. Quickstart: Deploy a python web app to azure app service, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [17] Microsoft Learn. Tutorial: Python app service using key vault and managed identity, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [18] Microsoft Learn. What are managed identities for azure resources?, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [19] Microsoft Learn. What is azure blob storage?, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [20] Pallets Projects. Flask documentation, 2026. Documentación oficial, consultado junio 2026.
- [21] Jorge Sanclemente Vilda. Soporte y estandarización de una arquitectura tecnológica de referencia para el despliegue de microservicios complejos mediante técnicas devops y gitops, 2024. Trabajo Fin de Grado, Universidad de Zaragoza.
- [22] SonarSource. Sonarqube documentation, 2025.